

Faculty : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranab, Anup Keshri and Supratim Biswas
Teaching Assistant : Swarna Aishwarya Twinkle (Research Scholar)

Lab Assignment 6

Objective : Semantic analysis and evaluation of an expression and an assignment statement. The grammar of Lab 5 is to be extended to handle two type of numeric values, integer and floating point numbers. Design of a parser and a semantic analyser to calculate the value of an expression in C is the first exercise. The basic analyser tool is to be augmented with more features.

Read the handout to examine how a lexer can effectively communicate with its parser in order to perform semantic actions along with parsing.

P1. Given the following ambiguous grammar to generate expressions in C involving 4 operators {+, −, /, *} and integer and floating point values as operands.

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid \text{int} \mid \text{real}$

You have to write lex and yacc scripts so that the generated parser behaves as described below. First write a C program to find out the value of the input expression given below. Compare the output of the C program with that generated by your parser.

Input Expression	Desired Output of the Parser
2.0 / 3 - 5 / 2.5 * 6 + 12.2	Value is : 2.000000 Value is : 3.000000 value of 2.000000 / 3.000000 is 0.666667 Value is : 5.000000 Value is : 2.500000 value of 5.000000 / 2.500000 is 2.000000 Value is : 6.000000 value of 2.000000 * 6.000000 is 12.000000 value of 0.666667 - 12.000000 is -11.333333 Value is : 12.200000 value of -11.333333 + 12.200000 is 0.866667

P2. Extend the set of operators given in P1 by the following binary ops., {<, <=, ==, !=, &&, ||}. Design a parser that is consistent with C semantics(see Appendix). A sample input and its desirable output is given as illustration.

Input Expression	Desired Output of the Parser
25 / 2 + 6 * 2 < 10 && 4 == 4	Value is : 25.000000 Value is : 2.000000 value of 25.000000 / 2.000000 is 12.500000 Value is : 6.000000 Value is : 2.000000 value of 6.000000 * 2.000000 is 12.000000 value of 12.500000 + 12.000000 is 24.500000 Value is : 10.000000 value of 24.500000 < 10.000000 is 0.000000 Value is : 4.000000 Value is : 4.000000 value of 4.000000 == 4.000000 is 1.000000 value of 0.000000 && 1.000000 is 0.000000

P3. Extend the grammar of P2 to include assignment statement of C. A sample input and its desirable output is given as illustration.

Input Expression	Desired Output of the Parser
myvariable = 2.0 / 3 - 5 / 2.5 * 6 + 12.2;	Value is : 2.000000 Value is : 3.000000 value of 2.000000 / 3.000000 is 0.666667 Value is : 5.000000 Value is : 2.500000 value of 5.000000 / 2.500000 is 2.000000 Value is : 6.000000 value of 2.000000 * 6.000000 is 12.000000 value of 0.666667 - 12.000000 is -11.333333 Value is : 12.200000 value of -11.333333 + 12.200000 is 0.866667 Value of myvariable is 0.866667

P4. The parsers generated in all the 3 problems are not strictly consistent with C semantics. The issue is related to the actions taken for a given subexpression. For example, a sub-expression of the form, 30 / 4, should produce the truncated integer value 7. In contrast the sub-expressions, 30.0 / 4.0, or, 30.0 / 7, or, 30 / 7.0, must yield the floating point value of 7.5. As a hint, you may use an attribute, E.type, that is to be set to 0 or 1 (for integer or real respectively), which is to be assigned as semantic actions with the rules.

$E \rightarrow \text{INT} \{ E.\text{type} = 0; E.\text{value} = \dots \}$ and similarly for $E \rightarrow \text{REAL} \{ \dots \}$

Construct such a parser for Problem P1, such that the value computed for a given expression by your parser will be identical to that produced by a C program for the same expression.

End of the Experiment 6

Precedence	Associativity	Arity	Operator	Function of the operator
13	Left	binary	*, /, %	multiplicative operators
12	Left	binary	+, -	arithmetic operators
10	Left	binary	<, <=,	relational operators
9	Left	binary	==, !=	equality, inequality
5	Left	binary	&&	logical AND
4	Left	binary		logical OR
2	Right	binary	=	assignment operators